INTERN TASK BRIEF

Task 3 — Structured Output, Memory & Streaming

Building on Strands Agents (Task 2)

Track: Agentic AI Development — Strands Agents SDK
Prerequisites: Task 1 (LLM Fundamentals) and Task 2 (Strands Agents, Parts A & B)
Estimated effort: 5 - days

Objective

In Task 2 you built working Strands agents, wired up built-in and custom tools, and got comfortable with streaming and basic session handling. Task 3 turns those building blocks into production-grade behaviour. You will make agents return data your code can trust, give them memory that survives across turns and restarts, and deliver responses to the user in real time.

Each of the three topics has a learning component and a hands-on component. Where a mini project is listed, the working code (with a short README) is the deliverable — not just notes.

At a Glance

Topic	Type	End Goal
1. Structured Output Response	Learning + Mini Project	An agent that returns validated, structured JSON responses
2. Memory & Sessions	Project Learning	Enhance the Copilot Agent so it remembers user choices
3. Streaming	Learning + Mini Project	An agent that streams responses in real time

Topic 1 — Structured Output Response

Learning + Mini Project

Why it matters

Free-text LLM output is fine for a human reader but fragile for code. The moment another function, API, or database consumes the model's answer, you need a predictable shape. Structured output makes the agent return data that conforms to a schema you define, so downstream code never has to parse prose.

What to learn

- Structured output from LLMs — what it is and when to use it instead of free text.
- Defining response schemas with Pydantic models (fields, types, defaults, nested models, descriptions).
- Using the Strands agent’s structured-output mechanism to bind a response to a Pydantic model.
- JSON schema enforcement — how the schema constrains the model and what happens when the model deviates.
- Response validation: catching missing or mistyped fields before they reach your code.

- Error handling: retries, fallback behaviour, and surfacing useful messages when validation fails.

Mini project — deliverable

Build an agent that always returns a validated, structured JSON response. Pick a small but realistic use case — for example, extracting fields from an unstructured note (a meeting summary, a support ticket, or a product review) into a typed object.

- Define a Pydantic model with at least 4–5 fields, including one nested object or list.
- Configure the agent to return that model as structured output.
- Add validation and error handling so a malformed response is caught and retried (or fails cleanly with a clear message).
- Demonstrate with at least three sample inputs, including one deliberately messy input.

Topic 2 — Memory & Sessions

Project Learning

Why it matters

An agent that forgets everything between turns can't hold a real conversation. Sessions let an agent retain context within and across interactions; memory lets it recall facts and user preferences over the longer term. This is what turns a one-shot prompt into an assistant.

What to learn

- Local session management — keeping conversation state in memory or on the local filesystem.
- S3 session management — persisting session state to Amazon S3 so it survives restarts and is shareable across instances.
- AgentCore memory / session management — using Bedrock AgentCore's managed memory for short-term and long-term recall.
- The difference between session state (this conversation) and memory (durable facts the agent should remember later).
- When to choose local vs. S3 vs. AgentCore — trade-offs in durability, cost, and operational overhead.

Project — deliverable

Enhance the Copilot Agent so it remembers the user's choices. The agent should recall relevant preferences or earlier decisions and use them in later turns without the user having to repeat themselves.

- Add a session backend (start with local, then move at least one path to S3 or AgentCore).
- Persist a few meaningful user choices and prove they are recalled in a later turn — and after a restart.
- Write a short note on which backend you used, why, and what you'd change for production.

Topic 3 — Streaming

Learning + Mini Project

Why it matters

Waiting for a full response before showing anything feels slow. Streaming pushes the answer to the user token by token as it's generated, which dramatically improves perceived responsiveness — the standard UX for modern chat assistants.

What to learn

- Streaming responses from agents — how Strands exposes a stream of events instead of a single result.
- Token-by-token streaming and async iteration over the response stream.
- Partial output handling — accumulating chunks, handling tool-call events mid-stream, and detecting completion.
- UX patterns for streamed responses — showing progress, handling interruptions, and graceful error recovery mid-stream.

Mini project — deliverable

Build an agent that streams its responses in real time. A simple terminal or notebook interface is fine — the point is that output appears incrementally, not all at once.

- Stream a response token by token using async iteration.
- Handle the case where the agent calls a tool mid-stream (show that the stream resumes cleanly).
- Add basic error handling so a failure partway through doesn't leave the output in a broken state.

Submission Guidelines

- Submit working code for each mini project / project in a clearly named folder.
- Include a short README per task: what it does, how to run it, and what you learned.
- Keep your learning notes (the non-code parts) in a single document, organised by topic.
- Be ready to walk through your code and explain your design choices in the review.

Questions on scope or blockers? Raise them early — don't wait until submission day.